

Python 入门

中国人民大学

北京理工大学

- 全世界的编程语言大概有 600 多种。真正流行的编程语言就只有大概 20 种左右 (C, JAVA, Python)
- 2017 年发布的第 4 届顶级编程语言交互排行榜显示, Python 语言的流行度排名第

—

- C: 语言比较繁琐但是执行效率高
- Python: 语言简单易学但是执行效率低

- Matlab: 商用的软件, 价格不菲, 有一些专业性非常强的工具箱
- Python: 完全免费, 大部分常用功能在 Python 中可以找到相应的扩展库

- 为了使用 Python，我们推荐安装 Anaconda。Anaconda 是一款开源的软件，可以在 Linux、Windows 和 Mac OS X 上运行 Python，并可以快速安装 Python 的众多数据科学包以及依赖环境。
- 安装完 Anaconda，里面会有许多不同功能的软件。我们在接下来的章节中主要使用 Jupyter Notebook 作为编写 Python 代码的工具。Jupyter Notebook 是一种开源的网页应用程序，允许用户创建和共享包含实时代码、公式、可视化和叙述文本的文档，可以用于数据清洗与转换、数值模拟、统计建模、数据可视化、机器学习等。

本节主要介绍 Python 基础表达式用法：加法 +、减法 -、乘法 *、除法 /、指数 **、向下取整 //、取余 %，以及运算的优先级规则。

例如，加法、减法、乘法表达式由两个数字表达式之间的 +、-、* 号组成，比如说，

$4 + 5$

9

$4 - 5$

-1

$4 * 5$

20

乘方运算是通过两个连续的 * 号来表达的，注意，两个 * 号中间不能用空格。

```
2 ** 3
```

```
8
```

```
3 ** 2
```

```
9
```

```
2 ** 0.5
```

```
1.4142135623730951
```


Python 表达式遵循熟悉的优先级规则：优先计算括弧之内的表达式，之后是指数运算，然后计算乘除，最后再算加减。

```
(1 + 3)**2
```

16

```
1 + 3**2
```

10

```
(2 + 1) * 4 ** 2 - 1
```

47

除法有三种，第一种是常见的浮点除法。

```
4 / 5
```

0.8

- 在进行/ 运算时，除数不能为 0，否则会给出语法错误提示
- 任何两个数相除，即使被除数能被除数整除，计算结果一定是浮点数

```
4 / 2
```

2.0

第二种除法运算符 `//` 叫向下取整，即比商更小的整数值。如果商是正数，取整数位；如果商是带非零小数位的负数，取整数位减去 1 以后的结果；如果恰好被整除的话，返回一个整数值。

```
9 // 3
```

3

```
5 // 2
```

2

```
5 // 3
```

1

在进行相除取整时，两个相连的除号之间也不能有空格。

```
5 // -3
```

```
-2
```

```
-5 // 3
```

```
-2
```

```
-5 // -3
```

```
1
```

第三种除法运算符 % 是一种取余运算。注意是向下整除之后的余数

5 % 2

1

5 % 3

2

5 % -3

-1

-5 % 3

1

-5 % -3

-2

注意，任何两个数字相除，计算结果总是浮点数。

```
9 / 3
```

```
3.0
```

```
9 % 3
```

```
0
```

```
9 // 3
```

```
3
```

```
11 // 3 * 3 + 11 % 3
```

```
11
```

本节主要介绍 Python 数值类型：int（整数），complex（复数）和 float（浮点数），以及对应的数值精度。

整数表示没有小数部分的整数，包括负数、零或者正数。Python 可以处理任意大小的整数，但整数不能以 0 开头

```
3
```

```
3
```

```
0
```

```
0
```

```
-786
```

```
-786
```


在 Python 中，浮点数用小数点表示。与整数型没有小数点相比，浮点数总是带一个小数点。浮点数运算则可能会有四舍五入的误差。

```
0.0
```

```
0.0
```

```
5.
```

```
5.0
```

```
15.20
```

```
15.2
```

```
21.9
```

```
21.9
```

浮点数也可以使用科学计数法，格式为 $a \text{ e } b$ ，其中， a 、 b 为数值， e 是符号。

```
32.3e+18
```

```
3.23e+19
```

```
-32.54e100
```

```
-3.254e+101
```

```
70.2E-12
```

```
7.02e-11
```

浮点数非常灵活，但有一些限制。

- 浮点数可以表示很大或者很小的数字，但存在一定的范围限制
- 浮点数只能表示 15 位或者 16 位有效数字，剩下的精度就会丢失了
- 这些浮点数在做算术运算之后，最后的几位数字常常可能不准确。

第一个问题可以通过两种方式来观察。如果一个计算的结果是一个非常大的数字，那么它会被表示为无限大；如果是一个非常小的数字，则表示为零。

```
2e306
```

```
2e+306
```

```
2e306 * 10
```

```
2e+307
```

```
2e307 * 100
```

```
inf
```

接上页

```
2e-322
```

```
2e-322
```

```
2e-322 / 10
```

```
2e-323
```

```
2e-322 / 100
```

```
0.0
```

第二个问题可以通过涉及超过 15 位有效数字的表达式来观察。在进行任何算术运算之前，这些额外的数字被丢弃。

```
0.6666666666666666 - 0.666666666666666123456789
```

```
-1.1102230246251565e-16
```

```
0.6666666666666666 - 0.666666666666666123456789
```

```
0.0
```

第三个问题可以通过一个例子来说明。当两个表达式应该相等时，可以观察到第三个问题。例如，表达式 $2^{0.5}$ 计算 2 的平方根，但是该值的平方不会完全恢复成 2。

```
2 ** 0.5
```

```
1.4142135623730951
```

```
( 2 ** 0.5) * ( 2 ** 0.5)
```

```
2.0000000000000004
```

```
( 2 ** 0.5) * ( 2 ** 0.5) - 2
```

```
4.440892098500626e-16
```

上面的最终结果非常接近零，但不精确为零。这个问题几乎出现在所有的编程语言中，并不单单是 Python 的问题。

Python 支持复数，复数由实数部分和虚数部分构成， $a + bj$ ，或者 `complex(a,b)` 表示。注意：1、可以使用 `j` 或者 `J` 来表示虚数，这两种方法等价；2、虚数 `j` 或者 `J` 前面必须要有一个数值，不能单独使用 `j` 或者 `J`。

```
3.14j
```

```
3.14j
```

```
complex(0,3.14)
```

```
3.14j
```

```
2j
```

```
2j
```

```
5.j
```

```
5j
```



```
9.322e-36j
```

```
9.322e-36j
```

```
.876j
```

```
0.876j
```

```
-.6545+0j
```

```
(-0.6545+0j)
```

```
3e+26j
```

```
3e+26j
```

```
4.53e-7j
```

```
4.53e-7j
```

```
complex(2, 5.)
```

```
(2+5j)
```

本节主要介绍 Python 中的字符串数据类型及其相关字符串方法：连接 `+`、重复 `*`、索引 `[]`、格式化 `%` 等。

字符串是 Python 中常用的数据类型。我们可以使用半角状态下的单引号或双引号（`'` 或 `"`）来创建字符串。字符串既可以包含传统的数值型数据，也可以包括布尔值 `True` 或 `False`。表达式的含义取决于其结构和正在组合的值的类型。

```
"data" + "science"
```

```
'datascience'
```

加法是字符串连接运算符，将这两个字符串组合在一起。

```
"data" + " " + "science"
```

```
'data science'
```

星号 * 是重复操作。

```
'Data ' * 2
```

```
'Data Data '
```

单引号和双引号都可以用来创建字符串：'hi' 和"hi" 是相同的表达式。双引号通常是首选，因为他们允许在字符串中包含单引号。

```
"This won't work with a single-quoted string!"
```

"This won't work with a single-quoted string!"

函数 `str()` 返回任何值的字符串表示形式。

```
"That's " + str(1 + 1) + ' . '
```

```
"That's 2."
```

Python 使用方括号来截取字符串。从左到右，索引是从 0 开始，取 0, 1, 2 等，以此类推；从右到左，索引从 -1 开始，取 -1, -2, -3 等，以此类推。

```
'Hello'[0]
```

```
'H'
```

```
'Hello'[1]
```

```
'e'
```

```
'Hello'[-1]
```

```
'o'
```

接上页

```
'Hello'[0:2]
```

```
'He'
```

```
'Hello'[0:-2]
```

```
'Hel'
```

```
'Hello'[2:]
```

```
'llo'
```

```
'Hello'[:]
```

```
'Hello'
```

```
'Hello'[:-1]
```

```
'Hell'
```

用 in/not in 检验一个字符串是否整体包含在另一个字符串中。对于 in，如果字符串中包含给定的字符返回 True，否则为 False。比如，“H” in “Hello” 输出为 True；“M” not in “Hello” 输出为 True。

```
'Tom' in 'Tom and Jerry'
```

True

```
'Jerry' in 'Tom and Jerry'
```

True

```
'Spike' in 'Tom and Jerry'
```

False

对于字符串而言，我们可以使用字符串方法。字符串方法，本质上就是操作字符串的函数。这些方法通过在字符串后面放置一个点，然后调用该函数来调用。

```
"loud".upper()
```

```
'LOUD'
```

将第一个字母大写：

```
'statistics'.capitalize()
```

```
'Statistics'
```

将字符串居中，并使用空格将位数补齐到指定长度：

```
'statistics'.center(12)
```

```
' statistics '
```

可以尝试一下 `S.center(13)` 的效果。检验字符串是否至少含有一位字符，且所有字符均为英文字母：

```
'statistics'.isalpha()
```

True

方法 `replace` 替换字符串中的所有子字符串。`replace` 方法有两个参数，即被替换的字符串和替代字符串。

```
'hitchhiker'.replace('hi', 'ma')
```

'matchmaker'

字符串方法也可以使用变量名称进行调用，只要这些名称绑定到字符串。因此，例如，通过首先创建“ingrain” 然后进行第二次替换，以下两个步骤的过程从“train” 生成“degrade” 一词。

```
s = "train"
t = s.replace('t', 'ing')
u = t.replace('in', 'de')
u
```

'degrade'

注意 `t = s.replace('t', 'ing')` 的一行，不改变字符串 `s`，它仍然是“train”。方法调用 `s.replace('t', 'ing')` 只有一个值，即字符串“ingrain”。

s

'train'

我们简单介绍一下输入函数 `input`。执行 `input` 函数后，会等待用户键盘输入，直至按下回车键后自动退出。

```
input("按除回车键之外的其他任意键，显示输入的内容，直至按下回车键退出。")
```

接下来，我们介绍 `print` 函数。这个函数以后会经常调用。`print` 函数默认输出是换行的，如不换行需要在变量末尾加上逗号。`print()` 函数的语法是 `print(* objects, sep=' ', end='\n', file=sys.stdout)`。

- 第一个参数 `objects`，表示可以一次输出多个对象。输出多个对象时，需要用，分隔；
- 第二个参数 `sep` 用来间隔多个对象，默认值是一个空格
- 第三个参数 `end` 用来设定以什么结尾。默认值是换行符 `\n`，我们可以换成其他字符串；最后一个参数 `file` 表示要写入的文件对象

第一个例子是关于换行输出的。

```
print("What is your student #?")
```

What is your student #?

第二个例子是关于不换行输出的。

```
x = 'I love '  
y = 'Baymax.'  
print(x,y)
```

I love Baymax.

```
print(x, end = ' ')  
print(y)
```

I love Baymax.

第三个例子是关于以. 为分隔符的。

```
print("isbd","ruc","edu","cn", sep=".")
```

isbd.ruc.edu.cn

Python 支持字符串的格式化输出，即字符串中的一些元素可以用固定格式的操作符来控制显示。例如，%s 表示输出为字符串，%d 表示整数，%f 表示浮点数。

```
print("My name is %s and weight is %5.2f kg!" % ('Garfield', 50))
```

My name is Garfield and weight is 50.00 kg!

```
print("My name is %s and weight is %d kg!" % ('Garfield', 50))
```

My name is Garfield and weight is 50 kg!

print 函数中常用到的还有禁止转义字符 r/R, 即让 print 的内容按照字符串字面的意思来使用。

```
print(r'\n')
```

```
\n
```

```
print(R'\n')
```

```
\n
```

赋值语句

本节主要介绍 Python 变量及其命名规则、赋值语句用法，并列举几个例子进一步解释如何运用变量、赋值语句。

通过等号把变量值赋值给变量名。尚未赋值的变量名应该在等号左边，取值在等号右边。赋值语句的语法规则是这样的：

变量名 = 变量值

比如说，

```
a = 5
```

目前，a 的值就是 5。除非以后我们重新定义 a 的取值，否则 a 的值会保持不变。

```
a = 5
```

```
b = 6
```

```
a + b
```

之前赋值的名称可以在等号右边的表达式中使用。比如说,

```
c = a + 6  
d = c + 7  
d
```

18

```
quarter = 1/4  
half = 2 * quarter  
half
```

0.5

但是，仅仅是表达式的当前值赋给了当前的名称。如果这个值以后改变了，则由这个值定义的其他名称将不会改变取值。如果需要改变取值，需要重新赋值。

```
quarter = 4
```

```
half
```

0.5

```
half = 2 * quarter
```

```
half
```

8

变量名必须要用字母开头，可以包含字母和数字，并且区分大小写；名称不能包含空格，通常使用下划线 _ 来替换每个空格；变量名不能用数字开头或者包含空格；变量名不能使用保留字符。

```
var_1 = 2  
var_1
```

2

Python 可以给出中文变量名。

```
我  
= 3我  
+ 2
```

5

```
我  
/ 2
```

1.5

```
我  
// 2
```

1

```
我  
* 2
```

6

```
我  
** 2
```

9

通常会使用一些易于理解的变量名称。

```
purchase_price = 5
state_tax_rate = 0.075
county_tax_rate = 0.02
city_tax_rate = 0
sales_tax_rate = state_tax_rate + county_tax_rate + city_tax_rate
sales_tax = purchase_price * sales_tax_rate
sales_tax
```

0.475

可以通过使用 del 语句删除单个或多个对象：

```
del purchase_price, sales_tax_rate
```

作为一个例子，可以再来看看目前全球电影票房排行榜前 5 名的电影票房（亿美元）

复联之终局之战

=27.98阿凡达

=27.90泰坦尼克号

= 21.87星战之原力的觉醒

= 20.68复联之无限战争

= 20.48前五总票房

= 复联之终局之战 + 阿凡达 + 泰坦尼克号 + 星战之原力的觉醒 + 复联之无限战争前五总票房

118.91000000000001

Python 允许同时为多个变量赋值。

```
a = b = c = 1
```

三个变量被分配到相同的内存空间上。我们可以为多个对象指定多个变量。

```
a, b, c = 1, 2, 3
```

整型对象 1, 2 和 3 分别分配给变量 a, b 和 c。

Python 中的保留字符: 保留字符不能用作常数、变量, 或其他标识符。除了 False、None、True 之外, 保留字符只包含小写字母。

Table: 保留字符

and	except	not	assert	finally	or
break	for	pass	class	from	nonlocal
continue	global	raise	def	if	return
del	import	try	elif	is	with
while	lambda	yield	False	None	True
as	async	await	else	in	

多次传闻要上市的海底捞，这次来真的了。2018 年 9 月 26 日上午 9:30，海底捞创始人张勇、首席执行官杨利娟敲响了港交所的铜锣。从 1994 年创立到 2018 年，历史 24 年，海底捞终于被大家吃上市了。事实上，2018 年 5 月 17 日，港交所网站就披露了海底捞国际控股递交的上市申请材料。这家以服务体验出名的“中华火锅老大”首次晒出了全部家底。上市申请材料显示，海底捞 2015 年开设门店数量 146 家，2016 年开设门店数量 176 家，2017 年开设门店数量 273 家，2018 年预计新开设 180 家到 220 加新餐厅。因此，2018 年预计开设门店数量达到 453 家到 493 家。

```
Num2015, Num2016, Num2017 = 146, 176, 273
```

```
NumNew = 220
```

```
Num2018 = NumNew + Num2017
```

```
Num2018
```

493

```
Num2017 = 273
```

```
NumNew = 180
```

```
Num2018 = NumNew + Num2017
```

```
Num2018
```

453


```
Num2016/Num2015-1,Num2017/Num2016-1,NumNew/Num2017
```

```
(0.20547945205479445, 0.5511363636363635, 0.6593406593406593)
```

可以看出，近两年海底捞新开门店数量激增。当然，快速扩张可能会同时带来巨额流动负债。

2017 年人均消费 97.7 元，顾客总量达 1.06 亿人次，可以算得 2017 年海底捞的营业额为 103.56 亿元，这个数字接近海底捞公布 2017 年度营收额 104 亿元。

```
Sales = 1.06 * 97.7
```

```
Sales
```

```
103.56200000000001
```

海底捞在上市申请材料中公布的数据中，称 2017 年度每家门店平均每天有 1500 人造访。按照 2017 年总顾客人数 1.06 亿人、273 家门店、365 天计算来算的话，每家门店平均每天造访的人数约为 1064 人次左右；但是按照一年的工作日大概 250 天来算的话，则每家门店平均每天造访的人数可以达到 1553 人次。

```
1.06*1e8 / 273 / 365
```

```
1063.7764062421597
```

一年的有效工作日 250 天是怎么算出来的呢。

休息日

= 52 * 2法定节假日

= 11有效工作日

= 365 - 休息日 - 法定节假日有效工作日

250

$1.06 \times 10^8 / 273$ / 有效工作日

1553.1135531135533

2017 年餐厅 104 亿元营收额中，有 30 亿来自一线城市、52 亿来自二线城市、15 亿来自三线或三线以下城市，其他地区的营收额只有 7 亿。也就是锁，营收额的一半来自二线城市。这个信息大体能反映海底捞的顾客群。事实上，88.2% 的顾客都是回头客，6 城顾客每月去一次。

```
City2nd = 52/104  
City2nd
```

0.5

2017 年餐厅 104 亿元营收额，再加上 2 亿左右的外卖业务以及 0.3 亿的销售调味料和食材，总收入为 106.37 亿。

需要扣除原材料以及易耗品成本 43.13 亿，员工成本 31.2 亿，其他支出 20.1 亿，剩下的利润是 11.94 亿。收入的三成分给了员工。海底捞解决了接近 5 万人的就业，根据这个数字可以推算这 5 万员工的平均年薪在 6 万左右。

总收入

= 106.37 原材料

= 43.13 工资

= 31.2 其他

= 20.1 利润

= 总收入 / 原材料 / 工资 / 其他 —— 原材料比例

= 原材料总收入 / 工资比例

= 工资总收入 / 其他占比

= 其他总收入 / 原材料比例

0.40547146751903734

中国人民大学的前身是 1937 年诞生于抗日战争烽火中的陕北公学。中国人民大学现在应该可以进行多少年校庆呢？

校庆
= 2019 - 1937校庆

82

中国人民大学西郊校园 (中关村大街 59 号) 占地面积 906 亩，换算下来会有多少平方米、多少公顷呢？(1 公顷为 15 亩、1 亩等于 666.667 平方米)。

公顷数
= 906/15公顷数

60.4

1 公顷恰好是 1 万平方米，中国人民大学中关村校区面积大概为 60.4 公顷、即 60.4 万平方米。中国人民大学通州校区占地总面积 128 公顷，换算下来大约是 1920 亩地。

亩数
= 128*15亩数

1920

截至 2017 年 12 月，中国人民大学有专任教师 1884 人，其中教授 659 人，副教授 770 人。助理教授有多少人？教授、副教授、助理教授的比例？

专任教师

, 教授, 副教授 = 1884, 659, 770 助理教授

= 专任教师 - 教授 - 副教授 助理教授 专任教师 副教授 专任教师 教授 专任教师

/,/,/

(0.24150743099787686, 0.40870488322717624, 0.3497876857749469)

有两种可能性导致助理教授比例相对较低：一是青年教师数量确实不够；二是可能助理教授晋升为副教授的速度比引进助理教授的速度更快。

截至 2017 年 12 月，中国人民大学共有全日制在校生 24201 人，其中本科生 10759 人，硕士生 8479 人，博士生 3554 人，留学生 1409 人。

在校生

= 24201 本科生

, 硕士生, 博士生 = 10759, 8479, 3554 留学生

= 1409 硕士生

(+ 博士生) / 本科生

1.118412491867274

生师比

= 在校生专任教师 / 生师比

12.845541401273886

中国人民大学的研究生数量与本科生数量相比是 1.12 比 1，研究生略多一些。学生与教师的比例大概为 13:1，1 个教师指导约 13 个学生。关于这一点，我们稍微扩充一下，13 个学生的学费其实是远远不够支付 1 个教师的成本了，因此，学校运转需要大量国家拨款。这些都是公共资源，同学们应该要珍惜这种资源的来之不易。

中国人民大学有学校办公室、教务处、发展规划处等 26 个行政单位，统计学院、经济学院等 47 个院系，党委办公室、校工会、组织部等 11 个党群组织，图书馆、档案馆、信息技术中心、博物馆等 4 个教辅单位，7 个学生社团，18 个其他机构。

机构数

$= 26 + 47 + 11 + 4 + 7 + 18$ 机构数

113

中国人民大学下设 113 个机构。

赋值运算符

Python 中常用的赋值运算符。

```
a, b, c = 2, 4, 8
```

```
c = a + c
```

```
c
```

10

```
c += a
```

```
c
```

12

```
c *= a
```

```
c
```

24

```
c /= a
```

接上页

```
c %= a
```

```
c
```

```
0.0
```

```
c **= a
```

```
c
```

```
0.0
```

```
c //= a
```

```
c
```

```
0.0
```

我们把这些赋值运算符列在表2中。

Table: 赋值运算符

运算符	描述	实例
<code>+=</code>	加法赋值运算符	<code>c += a</code> 等价于 <code>c = c + a</code>
<code>-=</code>	减法赋值运算符	<code>c -= a</code> 等价于 <code>c = c - a</code>
<code>*=</code>	乘法赋值运算符	<code>c *= a</code> 等价于 <code>c = c * a</code>
<code>**=</code>	乘方赋值运算符	<code>c **= a</code> 等价于 <code>c = c ** a</code>
<code>/=</code>	除法赋值运算符	<code>c /= a</code> 等价于 <code>c = c / a</code>
<code>%=</code>	取余赋值运算符	<code>c %= a</code> 等价于 <code>c = c % a</code>
<code>//=</code>	向下取整赋值运算符	<code>c //= a</code> 等价于 <code>c = c // a</code>

位运算符是把十进制数字先换算成二进制，然后基于二进制来计算的，输出结果按十进制显示。我们以 $a = 60$ 和 $b = 11$ 两个数字来说明各种位运算。其中，变量 $a = 60$ 的二进制格式为 $a = 00111100$ ， $b = 13$ 的二进制格式为 $b = 00001101$ 。Python 中的位运算法则如下表。

Table: 位运算符

运算符	描述	实例
&	与：两个相应数位都为 1，返回 1，否则为 0	$a \& b = 12$ 二进制：00001100
	或：相应数位有一个为 1，返回 1，否则为 0	$a b = 61$ 二进制：00111101
^	异：两个相应数位不同时，返回 1	$a \wedge b = 49$ 二进制：00110001
~	数位取反：把 1 变为 0，把 0 变为 1	$\sim a = -61$ 二进制：11000011
<<	左移若干位：高位保留，低位补 0。但对其他大部分语言而言：高位丢弃，低位补 0	$a << 2 = 240$ 二进制：11110000
>>	右移若干位：低位丢弃，正数高位补 0，负数高位补 1	$a >> 2 = 15$ 二进制：00001111

注 1：取反运算符

当二进制数为正数时，其反码、补码和原码相同。当表示有符号数时，最高位为符号位，0 表示正数，1 表示负数。当二进制数为负数时，保持最高为符号位不变，将数值位按位求反 (得到反码)，然后在最低位加 1 得到补码。用到这个例子中时， $a = 0011\ 1100$ 。要注意，如果我们的系统是 16 位的，则 a 事实上是这么表达的。 $a = 0000\ 0000\ 0011\ 1100$ ，按位取反以后变成 $1111\ 1111\ 1100\ 0011$ 。此时的最高位是 1，表示负数。因此，按数值位按位取反，得反码是 $0000\ 0000\ 0011\ 1100$ ，然后再最低位加 1，变为 $0000\ 0000\ 0011\ 1101$ ，这个结果是 61。再带上负号，就变成 -61 了。

注 2: 左移和右移运算符 由于 Python 可以处理任意大的整数, 因此左移运算符不会把高位的数字 1 丢弃, 而是会继续保留。低位用 0 补齐。另外, 如果原始数据是负数, 转为二进制以后, 最高位是 1。此时, 对右移运算符而言, 高位应该用 1 而不是 0 补齐。

```
a = 60
```

```
b = 13
```

```
a&b
```

12

```
a|b
```

61

```
a^b
```

49

```
~a
```

-61

```
a << 2
```

240

要记住，如果我们的系统是 16 位的话，则事实上 a 被存储为 0000 0000 0011 1100。左移 2 位之后，变为 0000 0000 1111 0000，最高位是 0，即整数，取值为 $16 + 32 + 64 + 128 = 240$ 。

```
a >> 2
```

15

Python 语言支持逻辑运算符。如果赋值 $a = 60$ 且 $b = 13$ 的话，我们给出三种逻辑运算符以及运算结果。

Table: 逻辑运算符

运算符	逻辑表达式	描述	返回
and	a and b	如果 a 为 False，返回 False，否则返回 b 的值	13
or	a or b	如果 a 为 True，返回 a 的值，否则返回 b 的值	60
not	not a	如果 a 为 True，返回 False；反之，返回 True	False

接上页

```
a1, b1 = 1, 2
```

```
a1 and b1
```

2

```
a1 or b1
```

1

```
a2, b2 = 0, 2
```

```
a2 and b2
```

0

```
a2 or b2
```

2

Python 语言支持成员运算符，用来判断一个字符（串）是否在另外一个字符串中。
下表列出了两种成员运算符。

Table: 成员运算符

运算符	描述
in	a in b: 如果 a 在 b 中，返回 True，否则为 False
not in	a not in b: 如果 a 不在 b 中，返回 True，否则为 False

接上页

```
'Tom' in 'Tom and Jerry'
```

True

```
'Jerry' not in 'Tom and Jerry'
```

False

```
'3' in '123'
```

True

身份运算符用于比较两个对象的存储单元。注：id() 函数用于获取对象的内存地址。

Table: 身份运算符

运算符	描述
is	a is b: 类似 id(a) == id(b), 引用同一对象, 返回 True, 否则 False
is not	a is not b: 类似 id(a) != id(b), 引用不同对象, 返回 True, 否则 False

```
a = b = 8
```

```
a is b
```

True

```
a, b = 8, 18
```

```
a is b
```

False

is 与 == 区别: is 用于判断两个变量引用对象是否为同一个, == 用于判断引用变量的值是否相等。

```
a = [1, 2, 3]
b = a
b is a
```

True

```
b = a
b == a
```

True

```
id(a), id(b)
```

(4427182344, 4427182344)

我们只需要把 b 的赋值方式稍微改一下，就可以看出差别来了。

```
b = a [:]  
b is a
```

False

```
b = a [:]  
b == a
```

True

```
id(a), id(b)
```

(4427182344, 4427013256)

Python 包含了各种比较值的运算符。比较运算符的运行结果一般为布尔值。例如，

$3 > 1 + 1$ 。

```
3 > 1 + 1
```

True

值 True 表明这个比较是正确的；Python 已经证实了 3 和 $1 + 1$ 之间关系的这个简单事实。下面列出了一整套通用的比较运算符。

Table: 比较运算符

比较	操作符	结果为 True 例子	结果为 False 例子
小于	<	$2 < 3$	$2 < 2$
大于	>	$3 > 2$	$3 > 3$
小于等于	<=	$2 <= 2$	$3 <= 2$
大于等于	>=	$3 >= 3$	$2 >= 3$
等于	==	$3 == 3$	$3 == 2$
不等于	!=	$3 != 2$	$2 != 2$

一个表达式可以包含多个比较，并且为了使整个表达式为真，它们都必须为真。例如，我们可以用下面的表达式表示 $1 + 1$ 在 1 和 3 之间。

```
1 < 1 + 1 < 3
```

True

两个数字的平均值总是在较小的数字和较大的数字之间。

```
x, y = 12, 5  
min(x, y) <= (x+y)/2 <= max(x, y)
```

True

字符串也可以比较，他们的顺序是字典顺序。

```
"Dog" > "Catastrophe" > "Cat"
```

True

最后我们给出这些运算符的优先级。在运算过程中，计算机从优先级高的运算符开始执行。

Table: 运算符优先级

**	指数（乘方运算符）
~	数位取反
*,/,%,//	乘、除、取余、向下取整
+, -	加减
>>, <<	右移、左移位运算符
&	与位运算符
^,	异、或位运算符
<=, <, >, >=, ==, !=	比较运算符
=, %=, /=, //=, -=, +=, *=, **=	赋值运算符
is, is not	身份运算符
in, not in	成员运算符
not, and, or	逻辑运算符

调用函数

本节主要介绍 Python 中的常用的内置函数以及函数调用。我们主要介绍常用的数学函数和随机数函数等。Python 具有一些常用的内置函数。调用这些函数，首先由知道这些函数名称。先写函数名，然后是括号，再对括号里相应的变量进行赋值。

```
abs(-12)
```

12

接下来，我们调用四舍五入函数：

```
round(5-1.3)
```

4

最大值函数 max 可以接受任意数量的参数并返回最大值。例如，

```
max(2,7)
```

7

```
max(2,2+3,4,3+8)
```

11

`range()` 也是常用的内置函数之一。调用 `range(start, stop[, step])` 可创建一个整数列表，一般用在 `for` 循环中。关于 `for` 循环，以后会详细介绍。大家现在可以忽略 `for` 循环的语法规则。`start`: 计数从 `start` 开始，默认是 0。`stop`: 计数到 `stop` 结束，但不包括 `stop`。`step`: 步长，默认为 1。

```
for i in range(2):  
    print(i)
```

```
0  
1
```

```
for i in range(1,5,2):  
    print(i)
```

```
1  
  
3
```

注意，Python 默认的是换行输出。如果希望不换行输出，只需改成如下格式即可：

```
for i in range(2):  
    print(i, end=' ')
```

0 1

上一小节用到的这些函数都是直接可以调用的，比如 `abs` 和 `round`，但有很多函数都会存储在一个称为模块（module）的函数集合中，比如 `log(2,2)` 和 `sqrt(2)`。调用这些函数之前，我们需要事先导入相应的模块（module）。数学运算常用的函数基本都在 `math` 模块中。Python `math` 模块提供了许多针对浮点数的数学运算函数。要使用 `math` 函数必须先导入。

```
import math  
math.sqrt(1)
```

1.0

```
math.sqrt(9)
```

3.0

```
math.sin(1)
```

0.8414709848078965

Python math 模块中还有两个数学常量：

```
math.e
```

```
2.718281828459045
```

```
math.pi
```

```
3.141592653589793
```

math 模块中包括我们常用的三角函数：

Table: 常用三角函数

函数	描述
<code>acos(x)</code>	返回 x 的反余弦弧度值。
<code>asin(x)</code>	返回 x 的反正弦弧度值。
<code>atan(x)</code>	返回 x 的反正切弧度值。
<code>cos(x)</code>	返回 x 的弧度的余弦值。
<code>hypot(x,y)</code>	返回欧几里得范数 $\sqrt{x^2+y^2}$ 。
<code>sin(x)</code>	返回 x 的弧度的正弦值。
<code>tan(x)</code>	返回 x 的弧度的正切值。
<code>degrees(x)</code>	将弧度转换为角度，如 <code>degrees(math.pi/2)</code> ，返回 90.0。
<code>radians(x)</code>	将角度转换为弧度。

导入模块之后，大家可以使用 `dir` 函数来查看这个模块中包含的函数。

```
import math  
dir(math)
```

`dir()` 输出结果的太长了，所以就不在这里显示了。这里给出 `math` 模块中常用的函数。

Table: `math` 模块中常用的数学函数

函数	返回值（描述）
<code>ceil(x)</code>	返回数字的上入整数，如 <code>math.ceil(4,1)</code> 返回 5
<code>exp(x)</code>	返回 e 的 x 次幂，如 <code>math.exp(1)</code> 返回 2.718281828459045
<code>floor(x)</code>	返回数字的下舍整数，如 <code>math.floor(4.9)</code> 返回 4
<code>log(x)</code>	如 <code>math.log(math.e)</code> 返回 1.0， <code>math.log(100,10)</code> 返回 2.0
<code>pow(x,y)</code>	$x^{**}y$ 运算后的值
<code>round(x[,n])</code>	返回 x 的四舍五入值，n 代表小数点后的位数
<code>sqrt(x)</code>	返回 x 的平方根

Python `cmath` 模块包含了一些用于复数运算的函数，也在数学运算中经常调用。这里给出一些 `cmath` 调用的例子。

```
import cmath  
cmath.sqrt(-1)
```

$1j$

```
cmath.sqrt(9)
```

$(3+0j)$

```
cmath.sin(1)
```

$(0.8414709848078965+0j)$

```
cmath.log10(100)
```

$(2+0j)$

如果忘记了模块中函数的使用方法，可以用 `help` 函数来查看帮助文件。

```
help(math.log)
```

Help on built-in function log in module math:

```
log(...)
```

```
log(x, [base=math.e])
```

Return the logarithm of x to the given base.

If the base not specified, returns the natural logarithm (base e) of x.

示例中方括号中的参数表示可选参数，也就是说，如果不给出方括号中的参数的话，就使用默认值。

接上页

```
math.log(16,2)/math.log(2,2)
```

4.0

```
math.log(16,2)/math.log(2)
```

5.7707801635558535

```
math.log(16)/math.log(2)
```

4.0

Jupyter 笔记本可以帮助记住不同函数名称。编辑代码时，在输入名称的开头少数字母之后按 Tab 键，可以显示补全该名称的列表。例如，在 `math` 后面按 Tab 键，来查看 `math` 模板中所有的可用函数。随着函数名字的补全，选项列表的范围会不断缩小。试试在下面几个命令后面按 Tab 键。

```
math.log
```

```
<function math.log>
```

如果要了解函数的更多信息，可以在名称后面放置一个问号，比如

```
math.log?
```

Python operator 模块提供一些常用的逻辑比较以及算术运算的操作。

```
import math  
import operator
```

```
math.sqrt(operator.add(4,5))
```

3.0

```
import operator  
operator.lt (1,3)
```

True

事实上，这和下面的表达方式是等价的。

```
1 <= 3
```

True

`operator.lt(a,b)` 相当于判断 $a < b$

`operator.le(a,b)` 相当于判断 $a \leq b$

`operator.eq(a,b)` 相当于判断 $a == b$

`operator.ne(a,b)` 相当于判断 $a != b$

`operator.ge(a,b)` 相当于判断 $a \geq b$

`operator.gt(a,b)` 相当于判断 $a > b$

常用的随机数函数包含在 random 模块当中，

```
import random  
random.choice(range(10))
```

2

```
random.randint(0,1)
```

0

```
random.randrange(10, 100, 2)
```

56

注意，random.randrange(10, 100, 2) 相当于从 [10, 12, ...96, 98] 序列中获取一个随机数，在结果上与 random.choice(range(10, 100, 2)) 等效。

下面我们给出 random 模块中常用的函数。

Table: 常用随机数函数

函数	描述
choice(seq)	从指定序列中随机挑选一个元素
randrange([start,]stop [,step])	从指定范围内获取一个随机数
randint(start, stop)	从含 start 和 stop 范围内随机生成一个整数
random()	在 [0,1) 范围内随机生成一个实数
seed([x])	改变随机数生成器的种子 seed
shuffle(lst)	将序列的所有元素随机排序
uniform(x,y)	随机生成下一个实数，它在 [x,y) 范围内
sample(lst, k)	从序列 lst 中随机获取指定长度 k 的序列

在本章中，

- 学习了几种比较简单的数据类型：整数、浮点数和复数
- 学习了变量名的命名规则：变量名可以用字母开头或者直接用中文变量名；但不能使用数字开头，变量名中间不能出现空格，不能使用保留字符
- 通过一些案例学习了各种运算符以及优先级的差别

